MOBILE DEVELOPMENT
MIDTERM LABORATORY EXAMINATION

Dart Programming Fundamentals • Object-Oriented Programming

Name: Melitante, Joezainne Q.	**Section:** 3.8 BSIT
Date: November 10, 2026	**Score: _____ / 50**

---

1. PROJECT OVERVIEW

The Community Learning Center Management System is an integrated, console-based application developed using the Dart programming language.

The application is designed to help a community learning center organize and manage its learning resources, participant records, and borrowing and returning transactions.

Through a menu-driven interface, users can register learning resources, register participants, view records, borrow available resources, return borrowed resources, view transaction history, and search records using identification numbers.

The system accepts actual user input, validates operations, updates resource quantities, and displays organized records.

The application demonstrates variables, data types, console input and output, operators, conditional statements, loops, functions, error handling, classes, constructors, encapsulation, Lists, Maps, inheritance, abstraction, polymorphism, method overriding, and mixins.

2. PROJECT OBJECTIVES

The main objectives of the Community Learning Center Management System are:

1. To develop an integrated Dart console application for managing community learning center records.
2. To allow users to register and manage learning resources.
3. To allow users to register and view participant information.
4. To process borrowing and returning transactions.
5. To maintain accurate resource quantities during borrowing and returning operations.
6. To prevent duplicate identification numbers and invalid transactions.
7. To provide a searching function using resource and participant identification numbers.
8. To demonstrate the required Dart fundamentals and object-oriented programming concepts.
9. To implement proper input validation and error handling.

10. To develop an organized and understandable application using meaningful classes, methods, and data structures.

3. APPLICATION FEATURES

3.1 Add Learning Resource

Allows the user to register a learning resource by entering its identification number, name or title, category, total quantity, and replacement cost.

The system checks for duplicate identification numbers and validates the entered information before adding the resource.

3.2 Register Participant

Allows the user to register participants by entering their identification number, full name, and section or group information.

The system prevents duplicate participant identification numbers and rejects empty or invalid information.

3.3 View Learning Resources

Displays all registered learning resources, including their identification numbers, names, categories, total quantities, and available copies.

3.4 View Participants

Displays all registered participants and their corresponding information.

3.5 Borrow Learning Resource

Allows a registered participant to borrow an available learning resource.

Before processing the transaction, the system checks whether the participant exists, whether the resource exists, and whether the resource has available copies.

When borrowing is successful, the available quantity decreases by one and a borrowing transaction is recorded.

3.6 Return Borrowed Resource

Allows the user to return a borrowed resource using its transaction identification number.

The system checks the transaction status to prevent the same transaction from being returned twice.

When the return is successful, the available resource quantity increases by one.

3.7 View Transaction Records

Displays borrowing and returning transaction information, including transaction identification numbers, participant IDs, resource IDs, borrowing dates, return dates, and transaction statuses.

3.8 Search Records

Allows the user to search for learning resources and participants using their identification numbers.

3.9 Exit Application

Allows the user to terminate the application properly.

4. TECHNOLOGIES AND DEVELOPMENT ENVIRONMENT

The application was designed for the following development environment:

- **Programming Language:** Dart
- **Application Type:** Console Application
- **IDE:** Visual Studio Code
- **Operating System:** Windows 10
- **Required Software:** Dart SDK, Visual Studio Code, and Dart Extension for Visual Studio Code

The application is a Dart console program and does not use Flutter.

5. PROGRAM DESIGN AND CLASS STRUCTURE

The application uses Object-Oriented Programming to organize its records and operations.

5.1 CenterRecord (Abstract Parent Class)

CenterRecord is the abstract parent class representing the common structure of records in the community learning center.

It contains shared information such as identification numbers and names. It also defines shared methods that subclasses must implement.

Its purpose is to provide a common structure for different record types.

5.2 LearningResource Class

LearningResource inherits from CenterRecord.

It represents learning materials registered in the system, such as books, modules, and other educational resources.

It manages:

- Resource identification number
- Resource name or title
- Resource category
- Total copies
- Available copies
- Replacement cost

It also provides methods for checking availability, borrowing copies, returning copies, and displaying resource information.

5.3 Participant Class

Participant also inherits from CenterRecord.

It represents individuals registered in the learning center.

It manages:

- Participant identification number
- Full name
- Section or group
- Active borrowing information

It provides methods for displaying participant details and managing participant-related borrowing information.

5.4 BorrowTransaction Class

BorrowTransaction represents an individual borrowing transaction.

It stores:

- Transaction identification number
- Participant identification number
- Resource identification number
- Borrowing date
- Return date
- Transaction status

This class allows the system to track borrowing activities and determine whether a transaction is still active or has already been returned.

5.5 CommunityLearningCenter Class

CommunityLearningCenter is responsible for managing the main operations of the application.

Its responsibilities include:

- Registering learning resources
- Registering participants
- Displaying records
- Processing borrowing transactions
- Processing returning transactions
- Searching records
- Managing Lists and Maps

It connects the different classes and ensures that the application works as one integrated system.

5.6 Timestamped Mixin

Timestamped is a reusable mixin that provides timestamp-related behavior.

It allows classes to reuse creation-time information without requiring another inheritance relationship.

The mixin demonstrates code reusability and the use of Dart mixins.

6. DART PROGRAMMING CONCEPTS IMPLEMENTED

6.1 Variables and Data Types

The application uses appropriate Dart data types to store and process information.

- **String:** Used for identification numbers, names, titles, categories, and section information.
- **int:** Used for resource quantities and other whole-number values.
- **double:** Used for replacement costs and monetary values.
- **bool:** Used for availability and transaction status.

The application also uses inferred variables where appropriate.

6.2 Console Input and Output

The application accepts user input through the console using Dart's input/output functionality.

It displays application titles, menus, instructions, prompts, records, confirmation messages, and error messages.

6.3 Operators

Arithmetic and comparison operators are used to process application rules.

Examples include decreasing available copies after successful borrowing, increasing available copies after successful returning, comparing identification numbers, checking whether the available quantity is greater than zero, and validating numerical values.

6.4 Conditional Statements

The application uses if/else statements to validate information and determine whether an operation can proceed.

Examples include checking duplicate IDs, validating resource availability, and verifying transaction status.

6.5 Switch Statements

A switch statement processes the user's main menu selection and directs the application to the corresponding operation.

6.6 Loops

The application uses a while loop to keep displaying the main menu until the user chooses to exit.

Loops are also used to display multiple records stored in collections.

6.7 User-Defined Functions

The application uses user-defined functions with parameters to perform specific operations.

Examples include functions for registering resources, registering participants, searching records, processing borrowing transactions, processing returning transactions, and validating user input.

These functions help organize the program and avoid unnecessary repetition.

6.8 Error Handling

The application uses try-catch and input validation to handle invalid or unexpected input.

Instead of terminating unexpectedly, the program displays understandable error messages and allows the user to continue using the menu.

6.9 Classes and Objects

The application uses classes as blueprints for creating objects.

For example, LearningResource objects represent registered learning materials, while Participant objects represent registered participants.

6.10 Constructors and Methods

Constructors initialize objects with the information required by each class.

Methods define the actions that objects can perform, such as displaying information, borrowing resources, and returning resources.

6.11 Encapsulation

Encapsulation is demonstrated through private fields using Dart's underscore naming convention.

Examples include _id, _name, _availableCopies, and _isActive.

Getters, setters, and methods provide controlled access to these fields.

This protects important information from uncontrolled modification.

6.12 Lists and Maps

Lists and Maps are used to store and organize application records.

Lists maintain collections of resources, participants, and transactions.

Maps store records using identification numbers as keys, allowing the application to retrieve records efficiently.

6.13 Inheritance

Inheritance is demonstrated through the LearningResource and Participant classes, which extend the abstract CenterRecord class.

This allows the subclasses to reuse common fields and behavior.

6.14 Abstraction

The abstract CenterRecord class provides a common structure for different record types.

It defines shared requirements without being directly instantiated.

6.15 Method Overriding

LearningResource and Participant provide their own implementations of shared methods inherited from CenterRecord.

For example, each subclass implements displayDetails() according to the information it needs to display.

6.16 Polymorphism

Polymorphism allows different subclass objects to be handled using the common CenterRecord parent type.

Although LearningResource and Participant share the same parent class, each displays its own specific information.

6.17 Mixins

The Timestamped mixin provides reusable timestamp behavior to classes that need creation-time information.

This demonstrates how Dart supports code reuse without requiring additional inheritance.

7. TESTING PLAN AND EXPECTED RESULTS

The following test scenarios are designed to verify the application's functionality.

Important: These are expected results, not claims of completed testing. Actual results must be obtained by running the application.

TEST 1: SUCCESSFUL LEARNING RESOURCE REGISTRATION

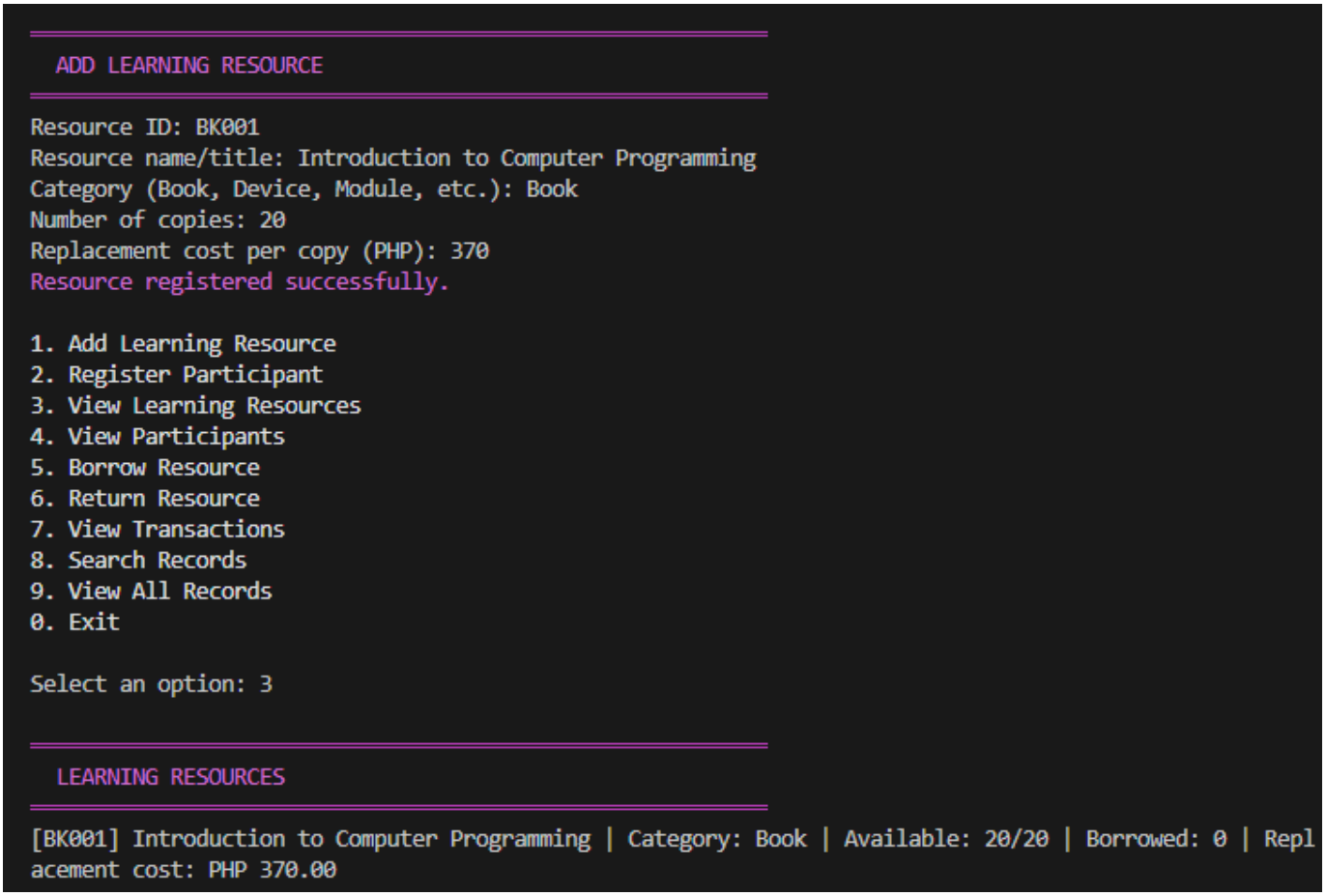
Objective:

To verify that the application can successfully register a learning resource.

Procedure:

1. Run the application.
2. Select Add Learning Resource.
3. Enter a unique resource ID.
4. Enter the resource name and category.
5. Enter the total number of copies.
6. Enter the replacement cost.
7. View the learning resources.

Result:



TEST 2: SUCCESSFUL PARTICIPANT REGISTRATION

Objective:

To verify that the application can register participants.

Procedure:

1. Select Register Participant.
2. Enter a unique participant ID.
3. Enter the participant's full name.
4. Enter the section or group.
5. View the participant records.

Result:

```
  COMMUNITY LEARNING CENTER MANAGEMENT SYSTEM

  Organizing resources, participants, and learning transactions.

1. Add Learning Resource
2. Register Participant
3. View Learning Resources
4. View Participants
5. Borrow Resource
6. Return Resource
7. View Transactions
8. Search Records
9. View All Records
0. Exit

Select an option: 2

  REGISTER PARTICIPANT

Participant ID: P001
Participant full name: Joezainne Melitante
Section/Group: 3.8 BSIT
Participant registered successfully.

1. Add Learning Resource
2. Register Participant
3. View Learning Resources
4. View Participants
5. Borrow Resource
6. Return Resource
7. View Transactions
8. Search Records
9. View All Records
0. Exit

Select an option: 4

  REGISTERED PARTICIPANTS

[P001] Joezainne Melitante | Section/Group: 3.8 BSIT | Active borrows: 0
```

TEST 3: SUCCESSFUL BORROWING AND RETURNING

Objective:

To verify that the application can process borrowing and returning transactions correctly.

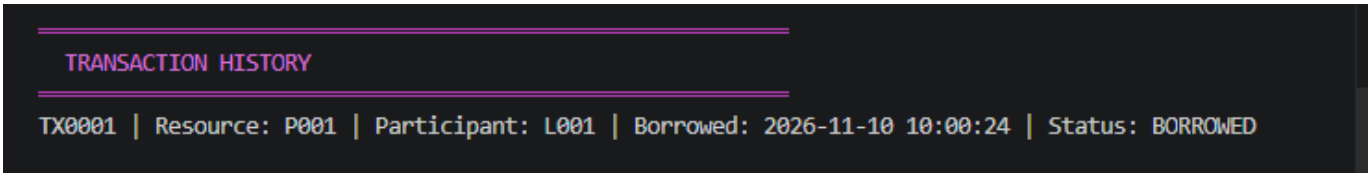
Procedure:

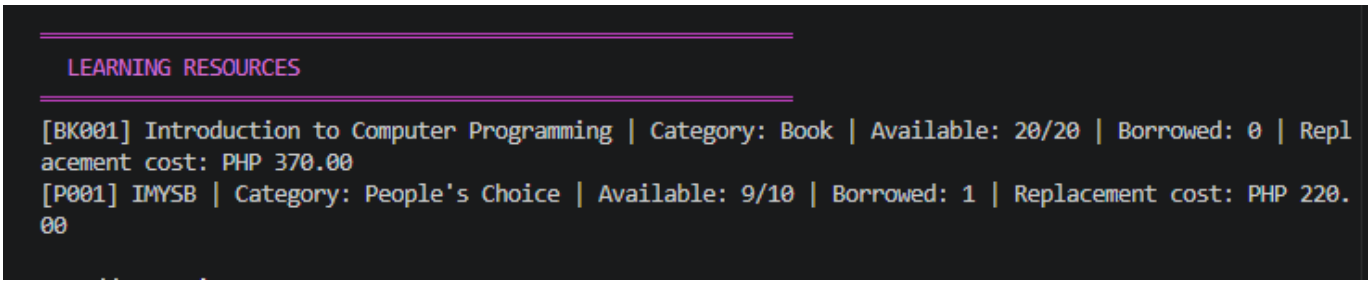
1. Register a learning resource with available copies.
2. Register a participant.
3. Select Borrow Resource.

4. Enter the registered resource ID.
5. Enter the registered participant ID.
6. Record the generated transaction ID.
7. Select Return Borrowed Resource.
8. Enter the transaction ID.
9. View the updated resource quantity and transaction records.

Result:







TEST 4: INVALID MENU INPUT

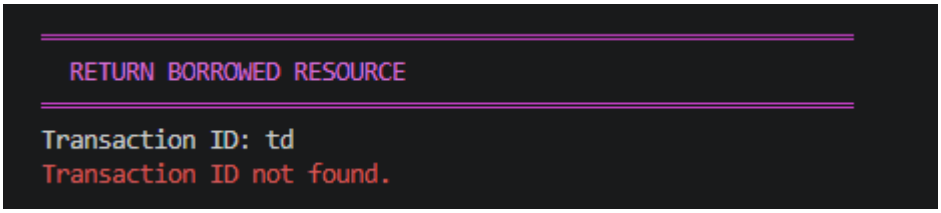
Objective:

To verify that the application handles invalid menu selections.

Procedure:

1. Run the application.
2. Enter a non-numeric menu selection such as abc.
3. Observe the displayed message.
4. Check whether the application returns to the main menu.

Result:



CONCLUSION

The Community Learning Center Management System is designed to provide an organized way of managing learning resources, participants, and borrowing transactions through one integrated Dart console application.

The application combines Dart programming fundamentals with object-oriented programming principles to create a structured and understandable system.

Through its registration, viewing, borrowing, returning, and searching features, the application demonstrates how programming concepts can be applied to a practical management scenario.

The use of classes, constructors, encapsulation, inheritance, abstraction, polymorphism, mixins, Lists, and Maps demonstrates the object-oriented programming concepts required by the laboratory examination.

The project also emphasizes input validation, error handling, and organized program flow to improve reliability and usability.

Before final submission, the application must be analyzed and executed using the installed Dart SDK in the Windows 10 and Visual Studio Code environment. Actual testing evidence must also be captured and included.